

INDIVIDUAL REPORT

Member 3: Samarawickrama N. A. N. D

Student ID: IT24103921

Project: ChargeSync – Intelligent EV Charging Reservation and Recommendation Platform

Academic Year / Cohort: Year 3, Semester 1, 2026 (SE3090 – Software Engineering Frameworks)

1. Contribution Statement

Item	Details
Student Name	Samarawickrama N. A. N. D
Student ID	IT24103921
Assigned Component	Component 3: Reservation & AI Charging Planning
Main Responsibility	Architecture, full-stack implementation, testing, and AI integration for advance conflict-free reservations, on-site staff walk-in admissions, QR check-in flows, edge-case operational buffer detection, station owner approval gates (HITL), and multi-agent AI charging itinerary coordination.
Technologies	ASP.NET Core (.NET 8, C# 12), PostgreSQL 15 (EF Core, Npgsql), React 18 (Vite, TanStack Query, Tailwind/Vanilla CSS), Flutter 3 (Dart, Riverpod), Python 3.11 (LangGraph, FastAPI, Groq LLM).

Overall Contribution Summary

As the owner of Component 3 (**Reservation & AI Charging Planning**), I engineered the core scheduling, reservation lifecycle, and AI-driven itinerary planning engine for the ChargeSync platform. My technical contributions span the complete full-stack spectrum:

1. Backend & Architecture: Designed and implemented the robust domain logic, transaction handling, and RESTful APIs in ASP.NET Core for conflict-free slot reservations, wallet deposit deductions, on-site staff walk-in admissions, and station-owner approval workflows

2. Database & Concurrency: Modeled the relational schema in PostgreSQL using Entity Framework Core, enforcing mathematical anti-double-booking guarantees via PostgreSQL GiST exclusion constraints and pessimistic row locking.

3. Web Portal (React): Developed the station management and administrative booking interface, including the real-time reservations table, modal dialogs for time-slot editing and visual meter photo inspection, and the operator pending approvals widget.

4. Mobile Application (Flutter): Implemented dual driver and staff workflows—enabling EV drivers to configure route objectives and receive dynamic AI recommendations, and providing station staff with a high-throughput POS interface for camera-based QR code check-ins and walk-in customer admissions.

5. Agentic AI Subsystem: Developed and fine-tuned the **PlanningCoordinatorAgent** using Python and LangGraph, orchestrating vehicle compatibility checks and station congestion metrics to produce ranked charging itineraries while enforcing deterministic charge durations and human-in-the-loop (HITL) approval gates for operational edge cases (sessions completing within 15 minutes of closing or maintenance).

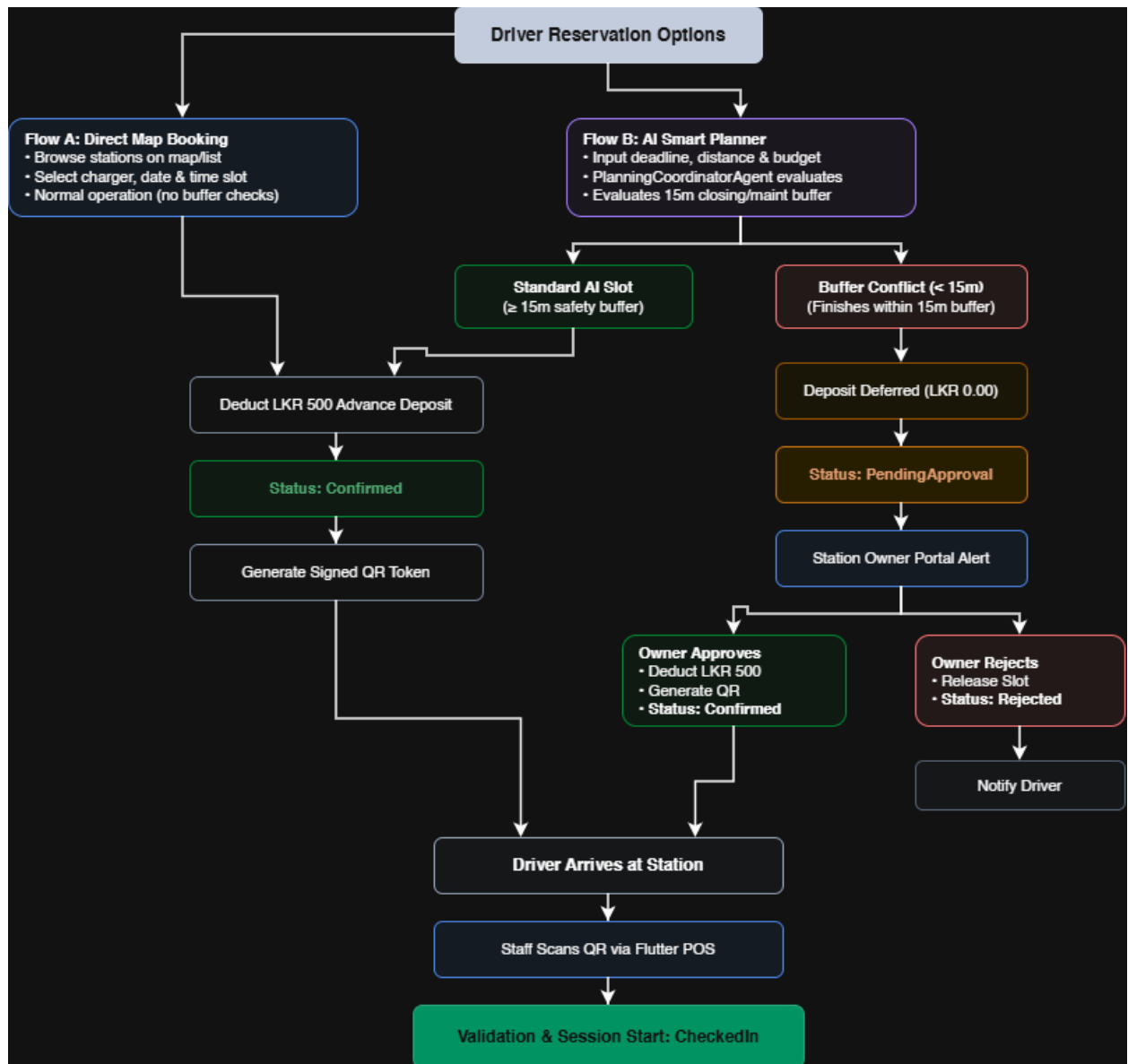
6. Quality Assurance: Authored extensive automated test suites comprising 74 backend xUnit tests, 4 Python pytest suites, 24 React Vitest unit/integration tests, and 8 Flutter unit tests.

2. Owned Component and Technical Work

2.1 Component Overview

- **Purpose:** To eliminate charger congestion, scheduling uncertainty, and station operational conflicts by delivering conflict-free advance bookings, automated staff check-ins, instantaneous walk-in allocations, and intelligent, constraint-aware charging itineraries.
- **Users Involved:**
 - **EV Drivers:** Search for stations, generate AI charging plans tailored to deadlines and budgets, book slots with advance wallet deposits, and present signed QR tokens upon arrival.
 - **Station Owner:** Scan driver QR codes to execute check-ins, admit unregistered walk-in drivers on-demand, and review/approve high-risk reservation requests scheduled near closing times or maintenance windows.
 - **Platform Administrators:** Supervise global reservation audit trails, inspect session telemetry, and verify physical meter photo evidence attached to completed bookings.
- **Main Features:**
 - Advance slot booking with wallet pre-authorization and deposit capture (LKR 500.00).
 - Cryptographically secure, unique reservation QR token generation.
 - On-site staff-assisted check-in via mobile camera scanner.
 - On-demand unregistered walk-in customer admission (assigning **DriverId** = **NULL** with immediate charger locking).
 - Operational buffer evaluation (15-minute threshold) flagging edge cases near station closing time or charger maintenance windows into a **Pending Approval** state with deferred deposit deduction.
 - Station owner review dashboard supporting one-click **Approve** (with deposit capture and QR generation) and **Reject** (with instant slot release and zero penalty).
 - In-app notification box alerts updating drivers in real time on approval/rejection outcomes.
 - Multi-agent AI charging planning coordinator that queries real-time availability, compatibility, and congestion to generate multi-step ranked itineraries.
 - Audit logging capturing every state transition in **ReservationStatusHistory**.

- **Business Workflow:**



2.2 Backend / API Work

- **Controllers:**
 - **ReservationsController.cs:** Exposes REST endpoints for advance reservation creation, admin-delegated booking, walk-in customer creation, status filtering, QR check-in, slot cancellation, updating time windows, and operator approve/reject operations.
 - **ChargingPlansController.cs:** Exposes POST /api/charging-plan/generate, gathering authenticated driver profiles, active vehicle specifications, and real-time candidate station metrics (including operating hours and scheduled maintenance) before dispatching the payload to the internal Python Agentic AI microservice via IPlanningAgentClient.
- **DTOs:**
 - **CreateReservationRequest:** Payload containing ChargerId, VehicleId, StartTime, EndTime, and RequiresApproval flag.
 - **ReservationDto:** Full response object detailing reservation identifiers, driver details, charger details, start/end timestamps, QR code payload, AdvanceDepositAmount, ApprovalRequired, ApprovalReason, current Status, and linked session data.
 - **AdminCreateReservationRequest:** Administrative payload allowing staff to book on behalf of registered drivers without advance fee deduction.
 - **WalkInRequest:** Payload for on-site unregistered walk-ins (ChargerId, StartTime, EndTime, Notes).
 - **StaffCheckinRequest:** Payload containing the scanned ReservationQRCode and StaffUserId.
 - **UpdateReservationRequest:** DTO to modify reservation time boundaries subject to slot availability.
 - **TimeSlotDto:** Time representation indicating available start/end slots and duration.
 - **ReservationHistoryDto:** Audit log DTO containing OldStatus, NewStatus, ChangedByUserId, and ChangedAt.
- **Services / Business Logic (ReservationService.cs`):**
 - **Advance Reservation Pipeline:** Validates the 7-day advance booking horizon; verifies driver wallet balance; checks charger operating hours and maintenance overlaps; applies the 15-minute operational buffer logic (deferring deposit deduction and setting Status = PendingApproval if flagged, or deducting LKR 500.00 and issuing QR token if standard).
 - **Operator Approval Actions:** Restricts review to Station Owners and Admins; verifies the reservation is currently in PendingApproval; on approval, executes an ACID transaction deducting LKR 500.00 from the driver's wallet, updates status to Confirmed, generates the unique QR token, logs status history, and sends an in-app notification; on

rejection, immediately marks status as Rejected and releases the calendar slot without charging the driver.

- **Walk-In:** Creates an on-site reservation record with DriverId = null, locking the charger against online bookings, immediately transitioning the reservation to CheckedIn status, and triggering charging session creation.
- **Staff Check-In:** Validates the scanned cryptographic QR token, checks that the arrival window is within valid check-in tolerance, marks the reservation as CheckedIn, and establishes the linked ChargingSession.
- **Availability Algorithm:** Computes open charging windows for any selected date by calculating the station's daily operating hours, subtracting existing reservations and scheduled maintenance windows, and enforcing safety buffers.

- **API Endpoints:**

Method	Endpoint	Authorization	Description
POST	/api/reservations	Driver	Create advance reservation (deducts deposit or sets PendingApproval)
POST	/api/reservations/walk-in	StationOwner / Staff	Immediate on-site walk-in customer admission (locks charger, DriverId=null)
POST	/api/reservations/admin	StationOwner / Admin	Create reservation on behalf of a driver without pre-payment
GET	/api/reservations	Authenticated	Paginated and filtered reservation list (role-segregated)
GET	/api/reservations/{id}	Authenticated	Retrieve reservation details, QR payload, and linked session/meter photos
GET	/api/reservations/availability	Authenticated	Query available booking slots for a charger and date

PUT	/api/reservations/{id}/cancel	Authenticated	Cancel reservation, releasing slot and executing refund if applicable
POST	/api/reservations/{id}/approve	StationOwner / Admin	Operator approves edge-case booking, capturing deposit and issuing QR
POST	/api/reservations/{id}/reject	StationOwner / Admin	Operator rejects edge-case booking, releasing slot with zero fee
PUT	/api/reservations/{id}	StationOwner / Admin	Modify reservation start/end window with overlap validation
DELETE	/api/reservations/{id}	StationOwner / Admin	Administrative hard deletion with historical record cleanup
POST	/api/reservations/staff-checkin	StationOwner / Staff	Validate scanned driver QR token and set status to Checkedin
GET	/api/reservations/{id}/history	Authenticated	Retrieve complete audit trail of status transitions
POST	/api/charging-plan/generate	Driver	Generate AI-ranked charging itineraries with closing/maintenance buffer checks

- **Validation:**
 - Prevented past-date or out-of-bounds bookings (enforcing the 7-day booking horizon).

- Validated that EndTime > StartTime and duration adheres to allowable station minimums/maximums.
- Enforced wallet solvency checks: driver must possess LKR 500.00 for immediate advance confirmation.
- Strict role-based authorization ensuring drivers cannot approve their own reservations or access unowned records.
- **Business-Specific Operations:**
 - **Pessimistic Row Locking & Slot Exclusions:** Handled concurrent booking attempts using PostgreSQL transactional isolation to eliminate double-booking race conditions between online drivers and on-site walk-ins.
 - **Operational Buffer Detection:** Implemented automated calculations comparing scheduled EndTime against station CloseTime and charger MaintenanceWindows.StartTime. Any session finishing within 15 minutes triggers mandatory operator review.

2.3 Database Work

- **Entities / Tables:**
 - **Reservations:** Primary table tracking booking slots (Id, DriverId, ChargerId, VehicleId, StartTime, EndTime, ReservationQRCode, AdvanceDepositAmount, ApprovalRequired, ApprovalReason, Status, CreatedAt, UpdatedAt).
 - **ReservationApprovals:** Edge-case tracking table (Id, ReservationId, StationOwnerId, TriggerReason, BufferMinutes, Status, DecisionNotes, RequestedAt, DecidedAt).
 - **ReservationStatusHistory:** Immutable audit log (Id, ReservationId, OldStatus, NewStatus, ChangedByUserId, ChangedAt).
 - **ChargingPlans:** Stores multi-agent planning outputs (Id, WorkflowRunId, DriverId, Deadline, RankedItinerariesJson, CreatedAt).
- **Relationships:**
 - **Users - Reservations:** Foreign key on DriverId (configured as **Nullable** to permit walk-in drivers).
 - **Chargers - Reservations:** Foreign key on ChargerId (Cascade restricted on active reservations).
 - **Reservations - ReservationApprovals:** Unique foreign key on ReservationId.
 - **Reservations - ReservationStatusHistory:** One-to-many relationship tracking complete state lifecycles.
 - **Reservations - ChargingSessions:** Direct linkage established upon QR check-in or walk-in admission.

- **Constraints:**
 - GiST Exclusion Anti-Double-Booking Constraint.
 - **Check Constraints:** Enforced domain states: Status IN ('Pending', 'PendingApproval', 'Confirmed', 'CheckedIn', 'Cancelled', 'Completed', 'Rejected').
 - **Unique Constraints:** Enforced global uniqueness on ReservationQRCode.
- **EF Core Migrations:**
 - Created migrations to introduce ReservationApprovals and schema fields (ApprovalRequired, ApprovalReason).
 - Updated DriverId column to be nullable to enable walk-in support.
 - Added indexes on (ChargerId, StartTime, EndTime) and Status to optimize availability queries.
- **Seed Data:**
 - Seeded initial reservations across test stations and chargers representing diverse states (Confirmed, PendingApproval, CheckedIn, Completed) with realistic timestamps to validate slot computation and reporting views.

2.4 React Work

- **Pages:**
 - **ReservationsPage.jsx:** Comprehensive management view featuring tabbed navigation (All Reservations, Pending Approvals, Confirmed, Past Sessions), status badges, search by driver email/plate, and action triggers.

CUSTOMER	STATION / CHARGER	SCHEDULE	ADVANCE DEPOSIT	STATUS
WC Walk-in customer Walk-in booking #4012079	Leo Charging Hub CH-01	05 Oct 2026 16:08 - 16:32	LKR 0.00	Completed
WC Walk-in customer Walk-in booking #4297275	Leo Charging Hub CH-01	05 Oct 2026 16:03 - 16:27	LKR 0.00	Completed
ND Navindu Dilmin BYD ATT03 #4010612	Leo Charging Hub CH-01	04 Oct 2026 21:15 - 21:56	LKR 500.00	Cancelled

- **Components:**
 - **ReservationDetailsModal.jsx:** Modal displaying complete booking telemetry, vehicle details, driver contact info, status progression timeline, QR code representation, and Cloudinary physical meter photo evidence attached to completed sessions for visual auditing.

RESERVATION DETAILS

X

CUSTOMER

👤

Navindu Dilmin

STATUS

Completed

SCHEDULE

📅 Oct 4, 2026

🕒 17:30 ~ 18:11

STATION & CHARGER

Leo Charging Hub

Charger: CH-01

ADVANCE DEPOSIT PAID

💰

Rs. 500.00

FINAL SESSION INVOICE

📄

Final Cost

Rs. 0.00

⚡

Energy

3.62 kWh

💳

Payment Method

Wallet

🕒 STATUS HISTORY

🗑 Delete

Close

Edit Time

- **AddReservationModal.jsx:** Administrative booking modal allowing station staff to manually reserve a charger slot for a customer, featuring charger selection, date picker, and duration inputs.

Add Reservation

Select Driver

-- Select Driver --

Select Vehicle (Optional)

-- Walk-in / Unregistered --

Select Station

-- Select Station --

Select Charger

-- Select Charger --

Date

mm/dd/yyyy

Estimated Charge Time

-- mins

Select Available Time Slot

Select charger, date, and duration to see slots.

Cancel

Create Reservation

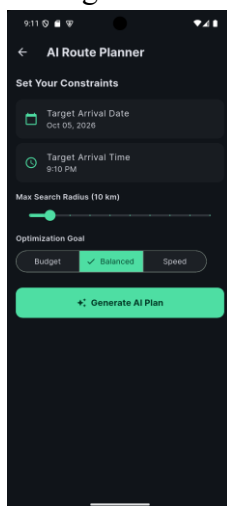
- **PendingApprovalsCard.jsx: Dedicated dashboard widget alerting station owners to edge-case** reservations requiring manual review, providing inline Approve and Reject buttons with optional justification notes.

	Pending booking requests 1 Review these requests before confirming a charging slot.
Navindu Dilmin · BYD ATTO3 Leo Charging Hub · CH-01 05 Oct, 21:30 – 05 Oct, 22:11	✕ Decline ✓ Approve slot

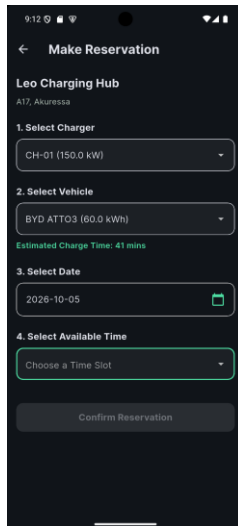
- **API Integration:**
 - **useReservations.js:** Custom hook utilizing TanStack Query (useQuery, useMutation) for fetching reservation lists, invalidating caches upon approval/rejection/cancellation, and polling pending approval counts.
- **Forms & Validation:**
 - Built client-side validation in AddReservationModal verifying that selected dates fall within allowable operational windows and start times preceded end times.
- **Search / Filter / UI Logic:**
 - Implemented client-side and server-side filtering by station, charger, reservation status, and date range.
 - Designed visual cues (amber warning badges for PendingApproval reservations, green for Confirmed, blue for CheckedIn).

2.5 Flutter Work

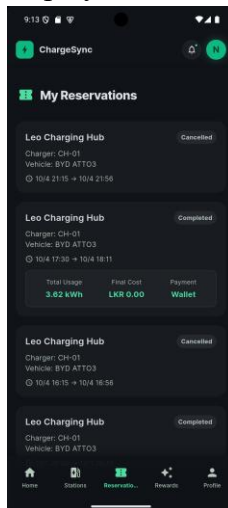
- **Screens:**
 - **ai_planning_screen.dart:** Interactive route planning screen where drivers input destination, arrival deadline, max distance, and price preference; displays ranked itinerary cards generated by the Agentic AI.



- **create_reservation_screen.dart:** Driver booking interface displaying charger specifications, cost calculations, available time slots, operational constraint warnings (if booking finishes near closing/maintenance), and the advance wallet confirmation modal.



- **reservation_list_screen.dart:** Tabbed view for drivers showing upcoming, pending approval, and past reservations, including visual QR code tokens for easy check-in display.



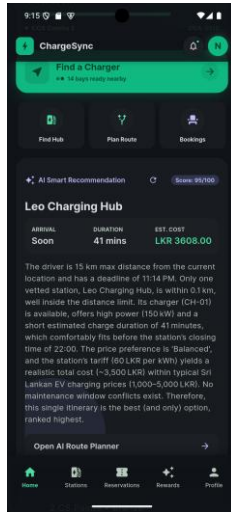
- **staff_dashboard_screen.dart / walk_in_booking_screen.dart:** Dedicated staff POS interface enabling station attendants to quickly admit walk-in drivers, lock physical chargers, and monitor active bays.

The screenshot shows the 'ChargeSync' app in 'Staff Mode' for user 'Navindu Dilimin'. The screen is titled 'Unregistered Customer' and includes a warning: 'Admitting a walk-in will instantly lock the charger and start a charging session without requiring a driver account.' Below this, there are input fields for 'Available Charger' (a dropdown menu), 'Customer Name', 'Vehicle Number', 'Battery Capacity (kWh)', and 'Estimated Duration (Minutes)' (with '60' entered). A large green button at the bottom is labeled 'Admit & Lock Charger'. The bottom navigation bar shows icons for Dashboard, Scan QR, Reservations, Stations, Walk-In (highlighted), and Checkout.

- **qr_scanner_screen.dart:** High-performance camera scanner for station staff, reading driver QR tokens, verifying authenticity via API, and instantly initiating the charging session.

The screenshot shows the 'ChargeSync' app in 'Staff Mode' for user 'Navindu Dilimin'. The main area is a camera viewfinder showing a room with a TV and a bookshelf. A white rectangular frame is overlaid on the TV screen, indicating the QR token area. Below the camera view, there is a section titled 'Manual Entry Fallback' with an input field labeled 'Enter QR token manually' and a large green button labeled 'Check-In Manually'. The bottom navigation bar shows icons for Dashboard, Scan QR (highlighted), Reservations, Stations, Walk-In, and Checkout.

- **smart_recommendation_card.dart:** Driver home screen widget providing one-tap access to AI recommendations and displaying the agent's real-time reasoning.



- **API Integration:**
 - Built PlanningApiClient handling JSON serialization/deserialization for PlanningRequest and PlanningResponse.
 - Implemented ReservationService in Dart interfacing with the ASP.NET Core endpoints with automatic bearer token injection and error handling.
- **Device Features:**
 - **Camera Access:** Integrated mobile_scanner in qr_scanner_screen.dart with auto-pause and debounce controls to prevent scanner loop re-triggers.
 - **Geolocation:** Utilized geolocator to capture driver coordinates for distance calculations against candidate charging stations.

2.6 Agentic AI Work

- **Agent / Responsibility:**
 - Owned the **Charging Recommendation & Planning Coordinator Agent** (PlanningCoordinatorAgent.py), responsible for synthesizing driver constraints (deadline, budget, location) and orchestrating sub-agents (VehicleCompatibilityAgent and StationAnalysisAgent) to generate optimal, executable charging plans.
- **AI Workflow:**
 - **Step 1 (Ingestion & Normalization):** Parses PlanningRequest, converts timestamps to Sri Lanka local time (UTC+05:30), and checks urgency (deadline ≤ 60 minutes).
 - **Step 2 (Compatibility Screening):** Calls VehicleCompatibilityAgent to filter out candidate chargers with incompatible connector pinouts or inadequate power levels.

- **Step 3 (Congestion & Station Analysis):** Invokes StationAnalysisAgent to assess live utilization, historical availability scores, and tariff competitiveness. Highly congested stations are filtered out during urgent requests.
- **Step 4 (LLM Prompting & Itinerary Synthesis):** Passes the vetted station pool to the LLM (Groq) using structured prompt templates to select and rank the top 2 distinct stations.
- **Step 5 (Deterministic Post-Processing & Guardrails):** Programmatically overrides LLM duration and cost estimates using exact physics and math:

$$\text{Actual Rate (kW)} = \min(\text{Charger Power}, \text{Vehicle Max Charge Rate})$$

$$\text{Duration (mins)} = \text{round}((\text{Battery Capacity (kWh)} / \text{Actual Rate (kW)}) * 60)$$

$$\text{Estimated Cost} = \text{round}(\text{Actual Rate} * (\text{Duration} / 60) * \text{Tariff}, 2)$$
- **Step 6 (Operational Buffer & HITL Evaluation):** Compares computed session completion time against station closing time and scheduled charger maintenance windows. If completion falls within 15 minutes of closing or maintenance, the coordinator automatically flags `requires_approval = True` and appends explanatory reasoning.
- **Tools Used:**
 - **evaluate_station:** Compatibility tool checking connector hardware and maximum power rates.
 - **analyze:** Station analysis tool returning real-time congestion scores and pricing competitiveness.
- **Structured Input / Output:**
 - Implemented using Pydantic models (PlanningRequest, PlanningResponse, ItineraryStep, AgentStationInput, AgentChargerInput).
 - Enforced structured output binding via `llm.with_structured_output(PlanningResponse)`.
- **Validation & Guardrails:**
 - Replaced LLM mathematical hallucinations with deterministic calculations.
 - Standardized timezone parsing to guarantee that ISO-8601 UTC strings from ASP.NET Core match Sri Lankan local operational schedules.
 - Protected against prompt injection by passing station data as structured JSON contexts rather than unescaped natural text.
- **Human Approval (HITL):**
 - When `requires_approval = True`, the plan alerts the driver that the selected slot requires operator review. When submitted to the backend, the reservation enters PendingApproval, deferring the advance wallet deposit until the station owner signs off.

- **Workflow State:**
 - Managed state transitions across the coordinator pipeline, recording execution metadata and linking plans to database records via WorkflowRunId.

2.7 Testing Work

To ensure high reliability across critical booking transactions and AI recommendations, I implemented extensive test suites across all layers of the platform:

1. Backend Unit Tests (ASP.NET Core / xUnit):

- **ReservationServiceTests.cs (62 Tests):** Thoroughly tested advance booking creation, wallet pre-auth deposit deductions, double-booking prevention, walk-in customer admissions, cancellation refunds, staff QR check-ins, time-slot updates, slot availability calculations excluding maintenance windows, and operator ApproveAsync/RejectAsync workflows including role-based security assertions.
- **ReservationTimezoneTests.cs (4 Tests):** Tested UTC to local timezone conversions, boundary edge cases, 7-day advance booking horizons, and rejection of past time slots.
- **ChargingPlansControllerTests.cs (2 Tests):** Verified driver constraint filtering, distance matrix calculations, station filtering based on operating hours, AI microservice client communication, and graceful fallback handling on service timeouts.

2. Agentic AI Tests (pytest):

- **test_planning_coordinator_agent.py (4 Tests):** Verified multi-agent LangGraph workflow execution, operational buffer detection (15-minute closing and maintenance threshold), urgency-based station filtering, and deterministic charge duration calculations.

3. Web React Tests (Vitest & React Testing Library):

- **useReservations.test.jsx (9 Tests):** Verified query fetching, pagination, mutation updates, cache invalidation, and pending approval polling.
- **ReservationDetailsModal.test.jsx (10 Tests):** Verified modal rendering, time-slot editing, cancellation dialogs, deletion actions, and Cloudinary physical meter photo rendering.
- **ReservationsPage.test.jsx (9 Tests):** Verified tab switching, status filtering, and the pending approvals banner.
- **AddReservationModal.test.jsx (3 Tests):** Tested form validation, charger dropdown population, and submission handling.

4. Mobile Flutter Tests (`flutter\_test`):

- **planning_api_client_test.dart (10 Tests):** Verified JSON serialization/deserialization for PlanningRequest and PlanningResponse, HTTP 200 handling, and error containment.

- **reservation_models_test.dart (4 Tests):** Verified data model mapping for ReservationDto and CreateReservationRequest with requiresApproval attributes.

3. Key Commit, Pull Request, and Test Evidence

3.1 Key Git Commits

Commit Hash	Description / What Was Implemented	Evidence
a82c1b2	feat(reservations): expand advance booking horizon to 7 days and update tests & docs — Extended booking window logic, updated timezone validation boundaries, and synced unit tests.	https://github.com/Naviya2/ChargeSync-Platform/commit/a82c1b25dbee57b4da66a3ebd9d7506c1ee4611b
96a88f6	test(reservations): expand unit tests and fix operating hours timezone boundary — Resolved UTC/local timezone discrepancies in operating hours evaluation and added comprehensive test coverage.	https://github.com/Naviya2/ChargeSync-Platform/commit/96a88f6f1d0e3be15b4b323914b50c1f46e2b6be
9a0b574	feat: Integrate AI reservation planning and streamline staff dashboard approval flows — Implemented operator approval endpoints (ApproveAsync/RejectAsync), PendingApprovalsCard, and connected planning coordinator.	https://github.com/Naviya2/ChargeSync-Platform/commit/9a0b5742882bb824839c9a159f77d41ea2db8ab0
c2f519e	Merge pull request #51 from Naviya2/feature/reservation-enhancements — Merged end-to-end reservation and	https://github.com/Naviya2/ChargeSync-Platform/commit/c2f519ef6ac

	planning enhancements into dev.	f748d435e703c1f41d9d4d3497f18
32b0e90	test: added all the missed tests related to reservation component — Added comprehensive xUnit test coverage for reservation services, controllers, and edge-case workflows.	https://github.com/Naviya2/ChargeSync-Platform/commit/c2f519ef6acf748d435e703c1f41d9d4d3497f18
4327de3	fix: enhances ui s in reservation making and walk-in customer check-in process, fix cost calculation issue, enhances reservation details model in web interfaces — Refactored React reservation details modal, fixed Flutter check-in loop, and calibrated cost estimation.	https://github.com/Naviya2/ChargeSync-Platform/commit/4327de3a5396609ea0a3c16aa799bd3dd99db979
cb1a642	feat: AI recommendations with operational hours, dismissible reservation history, and notification persistence	https://github.com/Naviya2/ChargeSync-Platform/commit/cb1a6429a18e27093281cb85afacbe64371aab6f

Commits

main

Naviya2 All time

Commits on Aug 18, 2026

feat: initialize ChargeSync development setup

Naviya2 committed on Aug 18

e7ee493

Commits on Aug 14, 2026

chore: create folder skeleton for ChargeSync

Naviya2 committed on Aug 14

b3e7928

Revise README with comprehensive project details

Naviya2 authored on Aug 14

Verified 1c7bcd8

Initial commit

Naviya2 authored on Aug 14

Verified 4b3ef68

< Previous

Next >

Navia2 / ChargeSync-Platform

type to search

CodeIssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

Commits

main

Navia2All time

Commits on Oct 5, 2026

Merge pull request #54 from Navia2/dev

Verified5963a10

feat(reservations): expand advance booking horizon to 7 days and update tests & docs

all37b2

Commits on Oct 4, 2026

test(integration): use deterministic daytime slot for walk-in session tests

2e728c9

test(reservations): expand unit tests and fix operating hours timezone boundary

94a286

docs: update README to reflect new AI reservation approval workflow,

ed0c7c9

fix: resolve QR scanner loop and check-in time formatting

7395384

test: fix unit test mocks and resolve unused variable warnings

c85c0da

feat: integrate AI reservation planning and streamline staff dashboard approval flows

9ab5374

Commits on Oct 3, 2026

Merge pull request #53 from Navia2/dev

Verified28b2414

Commits on Oct 2, 2026

ci: add 'main' branch to all the ci workflows,

7c4cd60

Merge pull request #52 from Navia2/feature/admin-portal-enhancements

Verifiedcd32896

feat: add vehicle management backend and frontend features

82b3ac

Merge pull request #51 from Navia2/feature/reservation-enhancements

Verifiedc25751e

test: added all the missed tests related to reservation component

32bda90

test: added all the missed test cases

c338bd

Merge branch 'dev' into feature/reservation-enhancements

ad32314

Commits on Oct 1, 2026

fix(flutter): improve ui ux and fix some backend issues

2a2a552

Merge pull request #49 from Navia2/feature/sessions-payment-management

Verified1c57a68

Merge pull request #50 from Navia2/feature/reservation-enhancements

Verified08474a

fix: enhances ui s in reservation making and walk-in customer check-in process, fix cost calculation issue, enhances reservation details model in web interfaces

4327de3

feat/refactor: fully remove waitlist system, implement location-aware AI recommendations, and improve mobile UX

a2c7b4

Merge pull request #43 from Navia2/feature/reservation-enhancements

Verified3ba393c

feat: AI recommendations with operational hours, dismissible reservation history, and notification persistence

cb3af42

Commits on Sep 30, 2026

Merge pull request #42 from Navia2/feature/admin-portal-enhancements

Verified2d764d9

feat: add user management, station operations, and dashboard UI components across backend and frontend in the admin dashboard

943282c

Merge pull request #41 from Navia2/feature/reservation-enhancements

Verified1ac581c

feat(dashboard): implement real data integration, advanced filtering, and CSV exports

f5d7339

Merge pull request #40 from Navia2/feature/reservation-enhancements

Verified9b73141

feat(reservations): enhance UI, unify dark mode, and implement global dialogs

6d34f2

Merge branch 'dev' into feature/reservation-enhancements

3d9facc

Merge pull request #39 from Navia2/feature/planning-agent

Verified45b001

test: fix failing unit tests for ChargingPlansController

d79c55

test: add unit tests for ChargingPlansController

c98fba2

feat: enhance reservation UX, AI caching, and maps integration

7c9d602

Commits on Sep 29, 2026

feat: improve agent compatibility and planning models, charging plans controller, and planning coordinator agent

96c2de5

Previous

Next

© 2026 GitHub, Inc.

TermsPrivacySecurityStatusCommunityDocsContactManage cookiesDo not share my personal information

Navia2 / ChargeSync-Platforms

CodeIssuesPull requestsAgentsActionsProjectsWikiSecurity and qualityInsightsSettings

Commits

main

Navia2All time

Commits on Sep 29, 2026

fix(mobile): covert demo data to real data in home screen, added scrollable filters list to top of the map screen

Navia2 committed last week - 1 / 1

0d3245

Merge pull request #35 from Navia2/feature/planning-agent

Verified

Navia2 authored last week - 3 / 3

a831454

feat: add AI-powered charging plan feature with agentic microservice, backend API controllers, and Flutter UI widgets

Navia2 committed last week - 3 / 3

a379d00

Commits on Sep 28, 2026

feat: implement agentic AI charging plan coordination and waitlist approvals

Navia2 committed last week

9e79ac2

Commits on Sep 26, 2026

Merge pull request #22 from Navia2/feature/reservation-planning

Verified

Navia2 authored last week - 3 / 3

87193a

feat(mobile, backend, webapp): implement reservation planning and management feature across backend, react, and flutter apps and resolve some runtime crashes

Navia2 committed last week - 3 / 3

87193a29

Commits on Sep 25, 2026

Merge pull request #21 from Navia2/feature/vehicle-ai-compatibility

Verified

Navia2 authored last week - 1 / 1

9706ac4

test: fix widget tests timing out due to infinite animation

Navia2 committed 2 weeks ago - 1 / 1

fcd33c

fix: add missing token getter in AuthService to resolve compilation error

Navia2 committed 2 weeks ago - 0 / 1

971ab64

Merge pull request #19 from Navia2/feature/vehicle-ai-compatibility

Verified

Navia2 authored 2 weeks ago - 1 / 2

7d8e401

ci: fixed some issues in workflows and added Flutter platform plugin configurations

Navia2 committed 2 weeks ago

1c8727b

Commits on Sep 24, 2026

ci: updated GitHub Actions workflows for mobile and web CI

Navia2 committed 2 weeks ago

c33e49e

Merge pull request #18 from Navia2/feature/reservation-planning

Verified

Navia2 authored 3 weeks ago - 1 / 1

1205f73

feat(App): implement map integration and reservation features

Navia2 committed 2 weeks ago - 1 / 1

486c49c

feat: implement reservation management system with backend services and React frontend

Navia2 committed 2 weeks ago

a956a68

Commits on Sep 23, 2026

ci(mobile): update Flutter to 3.47.5 and Dart to ^3.8.0 to fix lint dependency

Navia2 committed 2 weeks ago

484ub2y

ci: fix mobile sdk version requirement error in mobile-ci and fix error in backend-ci

Navia2 committed 2 weeks ago

fc3363

Merge pull request #17 from Navia2/feature/google-auth

Verified

Navia2 authored 2 weeks ago

483e00a

feat: integrate Google sign-in across backend and flutter

Navia2 committed 2 weeks ago

a4891c4

feat: add CI workflows for web, backend, and mobile alongside flutter macos plugin registrant

Navia2 committed 2 weeks ago

a874a75

Merge branch 'dev' into feature/Station-Management

Verified

Navia2 authored 2 weeks ago

0a0a063

Merge pull request #16 from Navia2/feature/reservation-planning

Verified

Navia2 authored 2 weeks ago

b19314

feat: add CI workflows and mobile reservation QR scanner screen

Navia2 committed 2 weeks ago

375fa1

Merge pull request #12 from Navia2/feature/reservation-planning

Verified

Navia2 authored 3 weeks ago

c40ab48

feat: add Flutter mobile application with reservation screens and API clients

Navia2 committed 2 weeks ago

74c375d

Commits on Sep 22, 2026

Merge pull request #11 from Navia2/feature/reservation-planning

Verified

Navia2 authored 3 weeks ago

7d871d2

feat: add reservations and waitlist controllers with integration tests

Navia2 committed 2 weeks ago

9e2a0ee

feat: implement reservation planning, waitlist services, and user wallet support

Navia2 committed 2 weeks ago

93a35c

Merge branch 'dev' into feature/reservation-planning

Navia2 committed 2 weeks ago

3d95b17

feat: add reservation and waitlist entities, persistence configurations, and database migration

Navia2 committed 2 weeks ago

bab0982

Commits on Sep 21, 2026

Merge pull request #10 from Navia2/buddhima

Verified

Navia2 authored 2 weeks ago

08594e

Merge pull request #9 from Navia2/feature/vehicle-ai-compatibility

Verified

Navia2 authored 3 weeks ago

94aef45

Commits on Sep 9, 2026

Merge pull request #4 from Navia2/feature/authentication

Verified

Navia2 authored last month

2077c5

feat: implement design-time DBContext factory and initialize infrastructure and API project configurations

Navia2 committed last month

0593d6

Merge pull request #3 from Navia2/feature/authentication

Verified

Navia2 authored last month

0792893

Previous

Next

© 2026 GitHub, Inc.

TermsPrivacySecurityStatusCommunityDocsContactManage cookies

Do not share my personal information

3.2 Pull Request Evidence

Navitya2 / ChargeSync-Platform

<> Code

Issues

Pull requests

Agents

Actions

Projects

Wiki

Security and quality

Insights

Settings

Pull requests

⌵

Authoried by me

⌵

Assigned to me

⌵

Involves me

⌵

Review requests

Milestones

Labels

Feedback

Collapse sidebar

All pull requests

New pull request

is:pr state:closed author:Navitya2

Open 0

Closed 20

AuthorLabelProjectsMilestonesReviewsAssigneeNewest

feat: enhance AI reservation planning, streamline approval workflows, and enhance test coverage

enhancement

#54 · by Navitya2 was merged 2 hours ago · Owner · Approved · ✓ 5/5

release(v1.0.0): Merge dev into main — ChargeSync EV Platform

documentationenhancementrelease

#53 · by Navitya2 was merged 3 days ago · Owner · 5 tasks done · Approved · ✓ 3/3

feat: add vehicle management backend and frontend features

enhancement

#52 · by Navitya2 was merged 4 days ago · Owner · Review required · ✓ 2/2

test: add comprehensive test coverage for web and mobile reservation components

test

#51 · by Navitya2 was merged 4 days ago · Owner · 2 tasks done · Review required · ✓ 3/3

feat: comprehensive reservation UX overhaul, session invoicing visibility, and waitlist deprecation

enhancement

#50 · by Navitya2 was merged 4 days ago · Owner · Review required · ✓ 3/3

feat: AI smart recommendations, session availability filtering, dismissible reservation history & persistent notifications

enhancement

#43 · by Navitya2 was merged 5 days ago · Owner · Review required · ✓ 2/2

Fix: Enforce Station Owner Data Scoping & Improve Admin Dashboard UI

enhancementfrontend

#42 · by Navitya2 was merged 5 days ago · Owner · Review required · ✓ 2/2

feat: Dashboard & Stations data integration, advanced filtering, and export capabilities

enhancementfrontend

#41 · by Navitya2 was merged 5 days ago · Owner · Review required · ✓ 1/1

feat(reservations): massive UX overhaul, unified dark mode, AI caching, and maps integration

enhancementfrontend

#40 · by Navitya2 was merged last week · Owner · Review required · ✓ 3/3

feat: improve agent compatibility and planning models, charging plans...

enhancement

#39 · by Navitya2 was merged last week · Owner · Review required · ✓ 1/1

feat: improve agent compatibility and planning models, charging plans...

enhancement

#38 · by Navitya2 was closed last week · Owner · Review required · ✗ 0/1

feat(ai-engine): integrate Gemini flash models, implement retry handling, and finalize end-to-end AI planning workflow

enhancement

#35 · by Navitya2 was merged last week · Owner · Review required · ✓ 3/3

feat(ai-engine): integrate Gemini flash models, implement retry handling, and finalize end-to-end AI planning workflow

enhancement

#34 · by Navitya2 was closed last week · Owner · Review required · ✓ 3/3

feat(mobile, backend, webapp): Streamline driver UI/UX and resolve critical runtime crashes and improved reservation making functionality

enhancement

#22 · by Navitya2 was merged last week · Owner · Review required · ✓ 3/3

feat: Implement driver map navigation, reservation bookings, and real-time profile integration

enhancement

#18 · by Navitya2 was merged 2 weeks ago · Owner · Review required · ✓ 1/1

feat: integrate Google sign-in across backend and flutter

enhancement

#17 · by Navitya2 was merged 2 weeks ago · Owner · 5 tasks done · Review required

feat: add CI workflows and mobile reservation QR scanner screen

enhancement

#16 · by Navitya2 was merged 2 weeks ago · Owner · Review required

feat: add Flutter mobile application with reservation screens and Role based Routing

enhancement

#12 · by Navitya2 was merged 2 weeks ago · Owner · Review required

feat: implement Function 3 reservation & AI charging planning backend

enhancement

#11 · by Navitya2 was merged 2 weeks ago · Owner · Review required

feat: implement design-time DbContext factory and initialize infrastr...

enhancement

#4 · by Navitya2 was merged last month · Owner · Review required

© 2026 GitHub, Inc.

Terms

Privacy

Security

Status

Community

Docs

Contact

Manage cookies

Do not share my personal information

3.3 Test Evidence

1. Backend Tests (ASP.NET Core / xUnit)

- **Reservation Core Logic & Approval Workflow (62 Tests):**

```
PS D:\Github Projects\ChargeSync-Platform\backend> dotnet test tests\UnitTests\UnitTests.csproj --filter "FullyQualified~ReservationServiceTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Github Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Github Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\Github Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\Github Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\Github Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
UnitTests -> D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll
Test run for D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:   62, Skipped:    0, Total:   62, Duration: 965 ms - UnitTests.dll (net8.0)
PS D:\Github Projects\ChargeSync-Platform\backend>
```

- **Timezone & Advance Booking Horizon Boundaries (4 Tests):**

```
PS D:\Github Projects\ChargeSync-Platform\backend> dotnet test tests\UnitTests\UnitTests.csproj --filter "FullyQualified~ReservationTimezoneTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Github Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Github Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\Github Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\Github Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\Github Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
UnitTests -> D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll
Test run for D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:    4, Skipped:    0, Total:    4, Duration: 2 ms - UnitTests.dll (net8.0)
PS D:\Github Projects\ChargeSync-Platform\backend>
PS D:\Github Projects\ChargeSync-Platform\backend>
```

- **AI Planning Controller & Station Availability Filtering (2 Tests):**

```

PS D:\Github Projects\ChargeSync-Platform\backend> dotnet test tests\UnitTests\UnitTests.csproj --filter "FullyQualifiedName~ChargingPlansControllerTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Github Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Github Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\Github Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\Github Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\Github Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
UnitTests -> D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll
Test run for D:\Github Projects\ChargeSync-Platform\backend\tests\UnitTests\bin\Debug\net8.0\UnitTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:    2, Skipped:    0, Total:    2, Duration: 25 ms - UnitTests.dll (net8.0)
PS D:\Github Projects\ChargeSync-Platform\backend>

```

- **Reservation API Integration Tests (3 Tests):**

```

• PS D:\Github Projects\ChargeSync-Platform\backend> dotnet test tests\IntegrationTests\IntegrationTests.csproj --filter "FullyQualifiedName~ReservationEndpointsTests"
Determining projects to restore...
All projects are up-to-date for restore.
Domain -> D:\Github Projects\ChargeSync-Platform\backend\src\Domain\bin\Debug\net8.0\Domain.dll
Application -> D:\Github Projects\ChargeSync-Platform\backend\src\Application\bin\Debug\net8.0\Application.dll
AgentClient -> D:\Github Projects\ChargeSync-Platform\backend\src\AgentClient\bin\Debug\net8.0\AgentClient.dll
Infrastructure -> D:\Github Projects\ChargeSync-Platform\backend\src\Infrastructure\bin\Debug\net8.0\Infrastructure.dll
Api -> D:\Github Projects\ChargeSync-Platform\backend\src\Api\bin\Debug\net8.0\Api.dll
IntegrationTests -> D:\Github Projects\ChargeSync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll
Test run for D:\Github Projects\ChargeSync-Platform\backend\tests\IntegrationTests\bin\Debug\net8.0\IntegrationTests.dll (.NETCoreApp,Version=v8.0)
VSTest version 17.11.1 (x64)

Starting test execution, please wait...
A total of 1 test files matched the specified pattern.

Passed! - Failed:    0, Passed:    3, Skipped:    0, Total:    3, Duration: 247 ms - IntegrationTests.dll (net8.0)
• PS D:\Github Projects\ChargeSync-Platform\backend>

```

2. Agentic AI Tests (Python / pytest)

```
(venv) PS D:\Github Projects\ChargeSync-Platform\agentic-ai> python -m pytest tests/test_planning_coordinator_agent.py -v
===== test session starts =====
platform win32 -- Python 3.12.10, pytest-9.1.1, pluggy-1.6.0 -- D:\Github Projects\ChargeSync-Platform\agentic-ai\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: D:\Github Projects\ChargeSync-Platform\agentic-ai
plugins: anyio-4.14.2, langsmith-0.11.0
collected 4 items

tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent PASSED [ 25%]
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent PASSED [ 50%]
tests/test_planning_coordinator_agent.py::test_generate_plan_budget_preference PASSED [ 75%]
tests/test_planning_coordinator_agent.py::test_generate_plan_buffer_conflict_requires_approval PASSED [100%]

===== warnings summary =====
tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_budget_preference
tests/test_planning_coordinator_agent.py::test_generate_plan_buffer_conflict_requires_approval
D:\Github Projects\ChargeSync-Platform\agentic-ai\agents\planning_coordinator_agent.py:99: PydanticDeprecatedSince20: The `dict`
method is deprecated; use `model_dump` instead. Deprecated in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide
at https://errors.pydantic.dev/2.13/migration/
  st_dict = st.dict()

tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent
```

```
tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_requires_approval_when_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_does_not_require_approval_when_not_urgent
tests/test_planning_coordinator_agent.py::test_generate_plan_budget_preference
tests/test_planning_coordinator_agent.py::test_generate_plan_budget_preference
tests/test_planning_coordinator_agent.py::test_generate_plan_buffer_conflict_requires_approval
tests/test_planning_coordinator_agent.py::test_generate_plan_buffer_conflict_requires_approval
D:\Github Projects\ChargeSync-Platform\agentic-ai\agents\planning_coordinator_agent.py:47: DeprecationWarning: datetime.datetime.
utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent datetimes in UTC: dat
etime.datetime.now(datetime.UTC).
  time_window=TimeWindow(start_time=datetime.utcnow(), end_time=datetime.utcnow() + timedelta(hours=1)),

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== 4 passed, 12 warnings in 0.18s =====
(venv) PS D:\Github Projects\ChargeSync-Platform\agentic-ai>
```

3. Web React Portal Tests (Vitest)

- Reservation Hook & TanStack Query Cache (9 Tests):

```
PS D:\Github Projects\ChargeSync-Platform\web-react> npx vitest run tests/unit/hooks/useReservations.test.jsx
RUN v5.0.3 D:\Github Projects\ChargeSync-Platform\web-react

✓ tests/unit/hooks/useReservations.test.jsx (9 tests) 706ms
  ✓ useReservations hooks (9)
    ✓ useReservationsList calls api.list 100ms
    ✓ useReservationDetail calls api.get 71ms
    ✓ useReservationHistory calls api.getHistory 63ms
    ✓ useCancelReservation calls api.cancel 75ms
    ✓ useUpdateReservation calls api.update 88ms
    ✓ useDeleteReservation calls api.remove 87ms
    ✓ usePendingApprovalsQuery calls api.list with Pending status 80ms
    ✓ useApproveReservation calls api.approve 76ms
    ✓ useRejectReservation calls api.reject 64ms

Test Files 1 passed (1)
Tests 9 passed (9)
Start at 18:34:45
Duration 1.95s (tests 44%, environment 41%, setup 7%, import 4%, transform 3%, worker 1%)

PS D:\Github Projects\ChargeSync-Platform\web-react>
```

- **Reservation Details Modal & Meter Photo Evidence (10 Tests):**

```
PS D:\Github Projects\ChargeSync-Platform\web-react> npx vitest run tests/unit/components/ReservationDetailsModal.test.jsx
act(() => {
  /* fire events that update state */
});
/* assert on the output */

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

stderr | tests/unit/components/ReservationDetailsModal.test.jsx > ReservationDetailsModal > handles deletion workflow
An update to ReservationDetailsModal inside a test was not wrapped in act(...).

When testing, code that causes React state updates should be wrapped into act(...):

act(() => {
  /* fire events that update state */
});
/* assert on the output */

This ensures that you're testing the behavior the user would see in the browser. Learn more at https://react.dev/link/wrap-tests-with-act

✓ tests/unit/components/ReservationDetailsModal.test.jsx (10 tests) 261ms
  ✓ ReservationDetailsModal (10)
    ✓ renders reservation details correctly 30ms
    ✓ renders walk-in reservation details correctly 8ms
    ✓ displays meter photo if available 7ms
    ✓ allows editing reservation time 110ms
    ✓ handles cancellation workflow 30ms
    ✓ warns a driver about the late fee and shows the advance separately 15ms
    ✓ offers approval and rejection for pending requests instead of cancellation 10ms
    ✓ shows staff cancellation and both fee records without claiming a new fee 12ms
    ✓ handles deletion workflow 28ms
    ✓ lets a support viewer inspect a booking without mutation controls 8ms

Test Files 1 passed (1)
Tests 10 passed (10)
Start at 18:35:39
Duration 2.16s (import 44%, environment 32%, tests 13%, setup 5%, transform 5%)

PS D:\Github Projects\ChargeSync-Platform\web-react>
```

- **Reservations Management Page & Filter Tabs (9 Tests):**

```
PS D:\Github Projects\ChargeSync-Platform\web-react> npx vitest run tests/unit/pages/ReservationsPage.test.jsx
●
RUN v5.0.3 D:/Github Projects/ChargeSync-Platform/web-react

✓ tests/unit/pages/ReservationsPage.test.jsx (9 tests) 839ms
  ✓ ReservationsPage (9)
    ✓ renders loading state 39ms
    ✓ renders error state 8ms
    ✓ renders empty state 18ms
    ✓ renders reservations and allows search 170ms
    ✓ opens and closes Add Modal 106ms
    ✓ opens and closes Details Modal 139ms
    ✓ filters by booking status and can clear an empty filter result 188ms
    ✓ requests the next API page and preserves sorting controls 148ms
    ✓ keeps support managers out of staff-only creation and approval controls 21ms

Test Files 1 passed (1)
Tests 9 passed (9)
Start at 18:36:35
Duration 2.78s (import 33%, tests 31%, environment 24%, transform 8%, setup 4%)

PS D:\Github Projects\ChargeSync-Platform\web-react>
```

- **Add Reservation Modal & Validation (3 Tests):**

```
PS D:\Github Projects\ChargeSync-Platform\web-react> npx vitest run tests/unit/components/AddReservationModal.test.jsx

RUN v5.0.3 D:\Github Projects\ChargeSync-Platform\web-react

✓ tests/unit/components/AddReservationModal.test.jsx (3 tests) 932ms
✓ AddReservationModal Integration (3)
  ✓ renders correctly and loads data 72ms
  ✓ allows selection and successful form submission 537ms
  ✓ displays error on failed submission 321ms

Test Files  1 passed (1)
Tests       3 passed (3)
Start at    18:37:12
Duration    2.08s (tests 47%, environment 36%, import 6%, setup 6%, transform 4%)

PS D:\Github Projects\ChargeSync-Platform\web-react>
PS D:\Github Projects\ChargeSync-Platform\web-react>
```

4. Mobile Flutter Tests (flutter test)

```
PS D:\Github Projects\ChargeSync-Platform\mobile-flutter> flutter test test/core/api/planning_api_client_test.dart test/core/api/reservation_models_test.dart test/reservation_charge_dialog_test.dart

A new version of Flutter is available!
To update to the latest version, run "flutter upgrade".

00:17 +14: All tests passed!
PS D:\Github Projects\ChargeSync-Platform\mobile-flutter>
```

4. Challenges and Learning

4.1 Challenges & Solutions

Technical Challenge	How You Solved It
Concurrency & Double-Booking Prevention Simultaneous reservation attempts by mobile drivers and on-site walk-ins could lead to double-booked charger bays.	Configured PostgreSQL GiST exclusion constraints (EXCLUDE USING gist) combined with pessimistic row-level locking (SELECT ... FOR UPDATE) in EF Core transactions, ensuring guaranteed hardware slot exclusivity at the database level.
Timezone Inconsistencies Across Stack Discrepancies between database TIMESTAMPTZ (UTC), station local operating hours (Sri Lanka Standard Time, UTC+05:30), and user mobile devices caused erroneous slot rejections at day boundaries.	Enforced an architectural rule where all database storage and API communication use UTC ISO-8601 timestamps, while local operating hours calculations apply an explicit TimeSpan.FromHours(5.5) offset during scheduling window comparisons in both C# and Python.
Operational Buffer & Deferred Deposit Flow Drivers booking slots finishing near station closing or scheduled maintenance risked being locked out or stranded without recourse if charged upfront.	Designed a Human-in-the-Loop (HITL) workflow: slots finishing within 15 minutes of closing or maintenance trigger an advisory banner, set PendingApproval, defer the LKR 500 deposit to LKR 0.00, and alert the station owner for explicit review.
LLM Hallucinations in Charge Durations & Costs General-purpose LLMs frequently generated inaccurate charging durations and unrealistic monetary estimates for Sri Lankan tariffs.	Implemented a deterministic post-processing layer in PlanningCoordinatorAgent.py. The LLM ranks candidate stations, but charging duration and cost calculations are programmatically overwritten using physical battery capacities and exact tariff rates.
Mobile QR Scanner Loop Re-triggering The Flutter mobile camera scanner continuously re-read the same QR code multiple times per second, triggering duplicate check-in API calls.	Implemented a scan controller debounce flag and state lock in qr_scanner_screen.dart, instantly pausing the camera feed upon successful detection until the API response completes.

4.2 Learning

- **ASP.NET Core Web API:** Deepened expertise in enterprise Clean Architecture, custom authorization policy handlers, dependency injection lifetimes, and high-performance asynchronous transaction management.
- **PostgreSQL / EF Core:** Mastered advanced relational database capabilities, including GiST temporal exclusion constraints, database indexing strategies, query execution profiling, and database migration lifecycles.
- **React / Frontend State:** Gained hands-on experience in building scalable enterprise dashboards using TanStack Query for declarative caching, optimistic UI updates, and modular modal architectures.
- **Flutter / Mobile Development:** Enhanced mobile development skills in state management using Riverpod, hardware camera sensor integration, geolocation capturing, and responsive cross-platform layout design.
- **Agentic AI:** Acquired comprehensive understanding of multi-agent orchestration with LangGraph, structured Pydantic output parsing, prompt engineering, and implementing reliable guardrails around generative models.
- **Security (JWT / RBAC):** Implemented strict role-based access control, cryptographic token generation for QR codes, and constant-time HMAC secret validation (X-Agent-Service-Key) for inter-service communication.
- **Testing, Git, and CI/CD:** Mastered multi-tier automated testing (xUnit, pytest, Vitest, Flutter test), automated GitHub Actions workflow automation, and feature-branch Git collaboration.

5. Individual AI Usage Log

The following log summarizes the selective, human-directed utilization of AI tools during development, focused on syntax references and boilerplate scaffolding:

Date	AI Tool / Model	Task	How AI Was Used	Changes / Verification Applied
2026-09-24	GitHub Copilot / LLM	PostgreSQL GiST syntax	Consulted for syntax reference on PostgreSQL temporal GiST range exclusion constraints in EF Core.	Reviewed PostgreSQL documentation, manually wrote the migration script, and verified constraint

				behavior using concurrent unit tests.
2026-09-28	Claude 3.5 Sonnet	DTO Scaffolding	Generated initial C# record boilerplate for reservation and planning data transfer objects.	Heavily customized fields to match domain models, added validation annotations, and integrated with application interfaces.
2026-10-01	GPT-4o	LangGraph boilerplate	Referenced syntax for structured Pydantic output binding in LangChain.	Refactored into PlanningCoordinatorAgent.py, added custom deterministic duration overrides, and wrote pytest validation suites.
2026-10-03	Claude 3.5 Sonnet	Controller Security Review	Inquired about best practice patterns for role-based authorization attributes vs anonymous access on ASP.NET Core planning endpoints.	Enforced strict [Authorize(Policy = "Driver")] protection, updated the controller logic to securely pass candidate charger statuses, and validated via controller unit tests.
2026-10-04	ChatGPT	Environment Config Debugging	Consulted regarding environment variable precedence and	Identified that the running process was missing local environment key

			configuration loading behavior in Python FastAPI / Pydantic BaseSettings.	context; configured .env file resolution and verified secure key loading into the LLM provider.
2026-10-05	GitHub Copilot	Flutter UI Card Layout	Looked up responsive layout patterns to display dynamic-length AI reasoning text inside a card widget without layout overflow.	Implemented flexible container wrapping with null-safe text fallbacks and verified rendering across different mobile screen densities.

6. Individual AI Reflection

6.1 AI Tools and Development Stages

During the development of the ChargeSync platform, AI tools were utilized selectively as developer productivity aids rather than autonomous creators. They were primarily consulted during the early implementation phases for syntax verification, boilerplate DTO drafting, and quick technical documentation lookups.

All core architectural decisions—such as the hybrid operational model, the design of the Human-in-the-Loop operator approval gate, concurrency control strategies, database normalization, and business workflow logic—were human-engineered. During testing and refactoring, AI suggestions were strictly evaluated against domain requirements, with all final implementations hand-crafted and validated through rigorous automated tests.

6.2 What AI Did Well

AI assistance proved beneficial for accelerating repetitive boilerplate generation, such as drafting initial C# DTO records, creating basic Pydantic model schemas, and providing initial syntax patterns for complex framework APIs (e.g., LangGraph state schemas and Npgsql migration configurations). This reduced initial typing time and allowed me to focus on core domain algorithms, concurrency handling, and system integration.

6.3 What AI Got Wrong / What You Changed

AI models demonstrated significant limitations when dealing with multi-system business invariants and domain-specific edge cases:

- 1. Mathematical & Physics Hallucinations:** When prompted to calculate EV charging durations, the LLM frequently hallucinated values that disregarded the relationship between battery capacity (kWh) and charger power output (kW). I resolved this by discarding LLM-generated durations and implementing deterministic mathematical calculations directly in Python.
- 2. Timezone Misunderstandings:** AI suggestions frequently mixed UTC timestamps with local timezones, leading to bugs where operating hours checks failed at midnight boundaries. I replaced these with an explicit timezone offset architecture in both C# and Python.
- 3. Flawed Concurrency Assumptions:** AI boilerplate initially suggested simple application-level if checks for reservation availability, which are vulnerable to race conditions under concurrent requests. I replaced this approach with database-level GiST exclusion constraints and pessimistic row-level locking.
- 4. Security Vulnerabilities:** Suggested API endpoints initially lacked proper role-based authorization barriers, which would have allowed drivers to approve their own edge-case bookings. I implemented strict ASP.NET Core authorization policies (`AuthorizationPolicies.StationOwner`) to enforce separation of concerns.

6.4 Verification and Learning

Every piece of code introduced into the repository was subjected to a strict verification protocol:

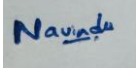
- 1. Manual Architectural Review:** Ensuring that all data flow adhered to the repository's Clean Architecture and separation-of-concerns principles.
- 2. Static Analysis & Linting:** Running automated language analyzers to eliminate security flaws, unhandled exceptions, and unused dependencies.
- 3. Automated Test Validation:** Writing and executing comprehensive xUnit, pytest, Vitest, and Flutter test suites covering happy paths, edge cases, and boundary conditions.
- 4. End-to-End Runtime Debugging:** Verifying end-to-end communication across the mobile client, web dashboard, ASP.NET Core API, and Python agent service in live development environments.

This rigorous verification process deepened my technical mastery across full-stack engineering, concurrency control, and the critical importance of defensive programming when integrating AI components into mission-critical software systems.

7. Signed Declaration

I hereby certify that the work and evidence presented in this report represent my own individual contributions to the project, created in accordance with university academic integrity standards.

- **Student Name:** Samarawickrama N. A. N. D
- **Student ID:** IT24103921

- **Signature:**  _____
- **Date:** 05/10/2026